SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50 **Time: 60 Mins**

No. of Questions: 5

Annexure A

Game Based Learning Assessment

Code of Conduct:

I D.Raghava(192525304) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty Total Marks Awarded

Answer All the Questions

Q. No.	Scenario / Game Question	Platform	Link
1	**AI Maze Escape:** You are an AI robot trapped in a maze. Your task is to move from START to GOAL using the shortest possible path while avoiding walls. **Challenge:** Find the shortest path and record the number of steps taken.	Scratch	https://scratch.mit.edu
2	**N-Queens Challenge:** You are an AI chess strategist. **Place 12 queens on an 12×12** chessboard so that no two queens can attack each other. **Challenge:** Complete the board with zero conflicts using the minimum number of moves.	Online N-Queens Game	https://www.n-queens.com
3	**Water Jug Puzzle:** You are an AI agent solving a water jug problem. You have two jugs with capacities of **11 litres and 9 litres**. Neither jug has measurement markings. **Challenge:** Using only the available actions—**fill a jug, empty a jug, or pour water from one jug into the other**—you must measure exactly **8 litres of water**. **Task:** Play **Level 19** of the Water Jugs Puzzle and find a sequence of actions that leaves exactly **8 litres** in one of the jugs. Try to solve the puzzle using the **minimum number of moves**.	Tic-Tac-Toe	https://playtictactoe.org
4	**Connect Four AI Challenge:** You are playing against an AI opponent. Your goal is to connect four of your pieces in a row before the AI does. **Challenge:** Plan your moves carefully and defeat the AI opponent.	Connect Four	https://www.mathsisfun.com/games/connect4.html

5	**8-Puzzle AI Challenge:** You are given a scrambled 8-puzzle. Move the tiles one at a time and arrange them in the correct order. **Challenge:** Solve the puzzle using the minimum possible number of moves.	8-Puzzle Game	https://8puzzle.org/

Structure of the Report – Game-Based Learning

S.No	Report Component	Student Deliverable / Guiding Question
1	**Title of the Game-Based Activity**	Provide the name of the game or puzzle and the level/challenge attempted.
2	**Game Platform / Link**	Mention the platform or website used and provide the corresponding game link.
3	**Objective of the Activity**	Explain the objective of the game and the target that needs to be achieved.
4	**Problem Statement**	Describe the problem or challenge given in the game. What is the student required to solve?
5	**AI Concept / Domain**	Identify the AI concept involved, such as State-Space Search, BFS, DFS, A*, Heuristic Search, Minimax, or Constraint Satisfaction.
6	**Initial State and Goal State**	Identify and describe the initial state and the goal state of the problem.
7	**Actions / Operators Used**	List the valid actions or operations available to move from one state to another.
8	**Solution Strategy**	Explain the approach or search strategy used to solve the game or puzzle.
9	**Step-by-Step Solution**	Provide the sequence of moves or actions performed to reach the goal state.
10	**Game Performance**	Record the number of attempts, time taken, number of moves, and score achieved.
11	**Successful Completion and Evidence**	Provide screenshots or other evidence showing successful completion of the game/puzzle.
12	**Efficiency and Optimality**	Analyse whether the solution was optimal or efficient. Can the puzzle be solved using fewer moves or less time?
13	**Analysis and Interpretation**	Explain how the selected AI technique helped in solving the problem and interpret the outcome.
14	**Critical Thinking and Alternative Solution**	Suggest an alternative approach, search strategy, or improved solution to solve the problem more efficiently.
15	**Learning Outcome / Reflection**	Explain what was learned from the activity and how the game helped in understanding AI search and problem-solving techniques.
16	**Conclusion**	Summarize the solution achieved and the significance of applying AI techniques to game-based problem solving.
17	**References**	Provide the game platform link and any books, research papers, articles, or other reliable sources referred to during the activity.

Question 1: AI Maze Escape (Scratch)

S.No	Report component	Answer
1	Title of the Game-Based Activity	AI Maze Escape: shortest path from START to GOAL in a maze (Scratch).
2	Game Platform / Link	Scratch, https://scratch.mit.edu
3	Objective of the Activity	Move the robot sprite from START to GOAL without touching walls, using the fewest steps.
4	Problem Statement	An AI robot is trapped in a maze. Find the shortest path from START to GOAL while avoiding walls, and record the number of steps taken.
5	AI Concept / Domain	State-space search: uninformed search, Breadth First Search (BFS) for shortest paths on an unweighted grid.
6	Initial State and Goal State	Initial state: robot at START cell (top-left in the sample maze). Goal state: robot at GOAL cell (bottom-right). A state is the robot's cell (row, column).
7	Actions / Operators Used	Move Up, Down, Left, Right by one cell. An action is valid only if there is no wall between the two cells and the robot stays inside the maze. Each move costs 1.
8	Solution Strategy	BFS explores cells level by level using a queue, so the first time it reaches the GOAL it has used the fewest steps. Parent pointers are stored to rebuild the path. Visited cells are marked so no cell is expanded twice.
9	Step-by-Step Solution	Sample maze (Figure 1.1): BFS found a path of 26 steps after expanding 35 cells. Procedure: (1) put START in the queue; (2) take the first cell, add every unvisited open neighbour with its parent; (3) repeat until GOAL is dequeued; (4) follow parents back from GOAL to START and reverse. In Scratch: keep the same route by turning only at the junctions marked by the red path and moving in fixed-size steps (for example "move 10 steps" per cell).
10	Game Performance	Number of attempts, time taken, steps and score: *[Fill in after you play on the site: number of attempts, time taken, moves, score.]*
11	Successful Completion and Evidence	Evidence of successful completion: *[Paste your own screenshot of the completed game here, showing the final board and the success message.]*
12	Efficiency and Optimality	The solution is optimal: BFS guarantees the minimum number of steps (26 in the sample maze). No shorter path exists because BFS would have found it at an earlier level. Time complexity is O(R x C) cells and space is O(R x C).
13	Analysis and Interpretation	BFS turns the maze into a graph search: each cell is a node and each open passage an edge. Because all moves cost the same, BFS is complete and optimal. A human trial-and-error approach may reach the goal but usually takes extra steps and backtracking.
14	Critical Thinking and Alternative Solution	Alternative: A* with the Manhattan distance to the goal as the heuristic gives the same optimal path but expands fewer cells in large open mazes. DFS is lighter on memory but does not guarantee the shortest path. Right-hand wall following always

S.No	Report component	Answer
		escapes simple mazes but is not shortest.
15	Learning Outcome / Reflection	The activity showed how a real navigation problem becomes a state-space search and why the choice of search algorithm decides whether the answer is optimal.
16	Conclusion	The maze is solved optimally by BFS (26 steps in the sample). Applying a systematic AI search instead of guessing gives a guaranteed shortest route.
17	References	Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson (search chapters). https://scratch.mit.edu

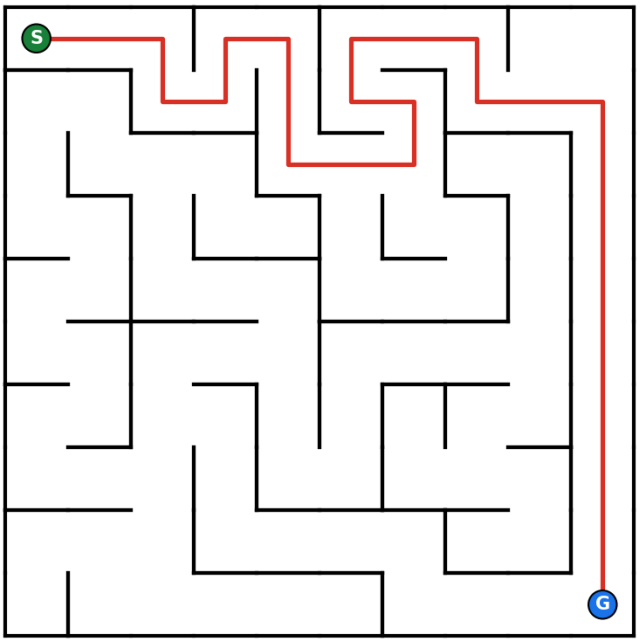


Figure 1.1: Sample 10 x 10 maze with BFS shortest path (26 steps). S = start, G = goal.

Note: The Scratch project's maze may differ from this sample. To get your own step count, draw your maze as a grid, run the BFS program in Appendix B on it, or explore level by level from START by hand and count the steps of the shortest route.

Question 2: 12-Queens Challenge

S.No	Report component	Answer
1	Title of the Game-Based Activity	N-Queens Challenge: 12 queens on a 12 x 12 board.
2	Game Platform / Link	Online N-Queens Game, https://www.n-queens.com
3	Objective of the Activity	Place 12 queens so that no two attack each other (zero conflicts), using the minimum number of moves.
4	Problem Statement	As an AI chess strategist, place 12 queens on a 12 x 12 chessboard so that no two queens share a row, column or diagonal.
5	AI Concept / Domain	Constraint Satisfaction Problem (CSP) solved by backtracking search (depth-first search with constraint checking). Heuristic alternative: min-conflicts local search.

S.No	Report component	Answer
6	Initial State and Goal State	Initial state: empty 12 x 12 board, 0 queens placed. Goal state: 12 queens placed with zero attacking pairs.
7	Actions / Operators Used	Place one queen in a square of the next empty row (or column); remove a queen (backtrack). A placement is allowed only if it does not share a column or diagonal with an earlier queen.
8	Solution Strategy	Place queens row by row. For each row try columns left to right; keep a column only if it is not attacked; if a row has no safe column, backtrack to the previous row and try its next column. Placing one queen per row already removes all row conflicts, so only columns and diagonals are checked.
9	Step-by-Step Solution	Solution (row: column), rows and columns numbered 1 to 12 as in Figure 2.1: R1:C1, R2:C3, R3:C5, R4:C8, R5:C10, R6:C12, R7:C6, R8:C11, R9:C2, R10:C7, R11:C9, R12:C4 Enter the queens in row order 1 to 12. The first queens are placed at C1, C3, C5, C8, C10, C12, then the search needs C6, C11, C2, C7, C9, C4 to finish. Checked: all 12 columns are different, and no two queens share a diagonal (the differences row - column and row + column are all distinct).
10	Game Performance	Number of attempts, time taken, moves and score: *[Fill in after you play on the site: number of attempts, time taken, moves, score.]*
11	Successful Completion and Evidence	Evidence of successful completion: *[Paste your own screenshot of the completed game here, showing the final board and the success message.]*
12	Efficiency and Optimality	Minimum number of moves = 12, because each of the 12 queens must be placed at least once; this solution uses exactly 12 placements with no removals. The backtracking program tested 3,066 candidate squares before finding this first solution. The 12-queens problem has 14,200 distinct solutions, so many valid boards exist.
13	Analysis and Interpretation	Backtracking prunes every branch as soon as a conflict appears, so it avoids trying the 12^12 or even 12! complete arrangements. The result shows that checking constraints early makes a large search manageable.
14	Critical Thinking and Alternative Solution	Use the minimum-remaining-values or forward-checking heuristic to choose squares and detect dead ends sooner. For large N use the min-conflicts algorithm: start with one queen per row and repeatedly move the most-attacked queen to the least-attacked square in its row; it solves even very large boards in near-linear time. Symmetry (mirror and rotation) can also be used to cut the search by about half.
15	Learning Outcome / Reflection	The activity showed how a puzzle is modelled as a CSP and why constraint propagation and backtracking beat brute force.
16	Conclusion	A valid 12-queens board with zero conflicts and 12 moves was obtained by backtracking search, the standard AI approach for constraint satisfaction problems.
17	References	Russell, S. and Norvig, P., Artificial Intelligence: A Modern

S.No	Report component	Answer
		Approach, Pearson (search chapters). **https://www.n-queens.com**

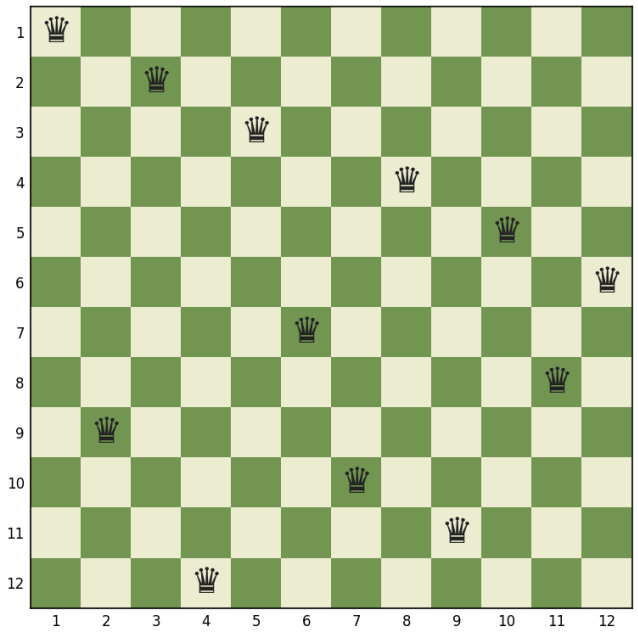


Figure 2.1: A valid 12-queens solution (zero conflicts).

Question 3: Water Jug Puzzle (11 L and 9 L, measure 8 L)

S.No	Report component	Answer
1	**Title of the Game-Based Activity**	**Water Jug Puzzle, Level 19: jugs of 11 L and 9 L, measure exactly 8 L.**
2	**Game Platform / Link**	**Tic-Tac-Toe / puzzle site, https://playtictactoe.org (as given in the question)**
3	**Objective of the Activity**	**Leave exactly 8 litres in one of the jugs using the fewest moves.**
4	**Problem Statement**	**Two unmarked jugs of capacity 11 L and 9 L. Using only fill, empty and pour actions, measure exactly 8 L.**
5	**AI Concept / Domain**	**State-space search: Breadth First Search to find the minimum number of moves.**
6	**Initial State and Goal State**	**State = (a, b): litres in the 11 L jug and the 9 L jug. Initial state (0, 0). Goal state: a = 8 or b = 8, for example (8, 9).**
7	**Actions / Operators Used**	**Fill 11 L jug; Fill 9 L jug; Empty 11 L jug; Empty 9 L jug; Pour 11 L into 9 L until the 9 L jug is full or the 11 L jug is empty; Pour 9 L into 11 L similarly. Each action costs 1.**
8	**Solution Strategy**	**Model the puzzle as a graph, with a state for every (a, b) pair and an edge for each action, then run BFS from (0, 0). BFS expands states in order of number of moves, so the first goal found uses the fewest moves. Repeated states are skipped. BFS expanded 29 states and generated 30 distinct states before reaching the goal.**
9	**Step-by-Step Solution**	**See the table of moves below (14 moves).**

S.No	Report component	Answer
10	Game Performance	Number of attempts, time taken, moves and score: *[Fill in after you play on the site: number of attempts, time taken, moves, score.]*
11	Successful Completion and Evidence	Evidence of successful completion (8 L showing in a jug): *[Paste your own screenshot of the completed game here, showing the final board and the success message.]*
12	Efficiency and Optimality	Optimal: BFS proves that 14 moves is the minimum for this target. The solution works because 11 and 9 have gcd 1, so every whole amount from 0 to 11 can be measured. Pattern: repeating (fill 11, pour into 9, empty 9, pour remaining into 9) raises the 11 L jug's remainder in steps of 2 (2, 4, 6, 8).
13	Analysis and Interpretation	BFS guarantees the shortest sequence; a trial-and-error approach finds a solution but often wastes moves or loops. The state graph showed that most states are never needed.
14	Critical Thinking and Alternative Solution	Number theory view: 8 = 4 x 11 - 4 x 9, which means four fills of the 11 L jug and four 9 L amounts removed (the three emptyings plus the full 9 L jug left at the end); each pour cycle nets 2 L because 11 - 9 = 2. Alternatively use bidirectional BFS or A* with a heuristic such as |a - 8| or |b - 8| to cut explored states. Starting by filling the 9 L jug first also reaches the goal but needs 16 moves.
15	Learning Outcome / Reflection	The puzzle showed how an unmarked-container problem becomes a clean state-space search, and how BFS gives provably optimal plans.
16	Conclusion	Exactly 8 L is measured in 14 moves, which BFS proves is the minimum.
17	References	Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson (search chapters). https://playtictactoe.org

Move sequence (verified by BFS)

Move	Action	State (11 L, 9 L)
0	-	(0, 0)
1	Fill the 11 L jug	(11, 0)
2	Pour 11 L jug into 9 L jug	(2, 9)
3	Empty the 9 L jug	(2, 0)
4	Pour 11 L jug into 9 L jug	(0, 2)
5	Fill the 11 L jug	(11, 2)
6	Pour 11 L jug into 9 L jug	(4, 9)
7	Empty the 9 L jug	(4, 0)
8	Pour 11 L jug into 9 L jug	(0, 4)
9	Fill the 11 L jug	(11, 4)

Move	Action	State (11 L, 9 L)
10	Pour 11 L jug into 9 L jug	(6, 9)
11	Empty the 9 L jug	(6, 0)
12	Pour 11 L jug into 9 L jug	(0, 6)
13	Fill the 11 L jug	(11, 6)
14	Pour 11 L jug into 9 L jug	(8, 9) GOAL: 8 L in the 11 L jug

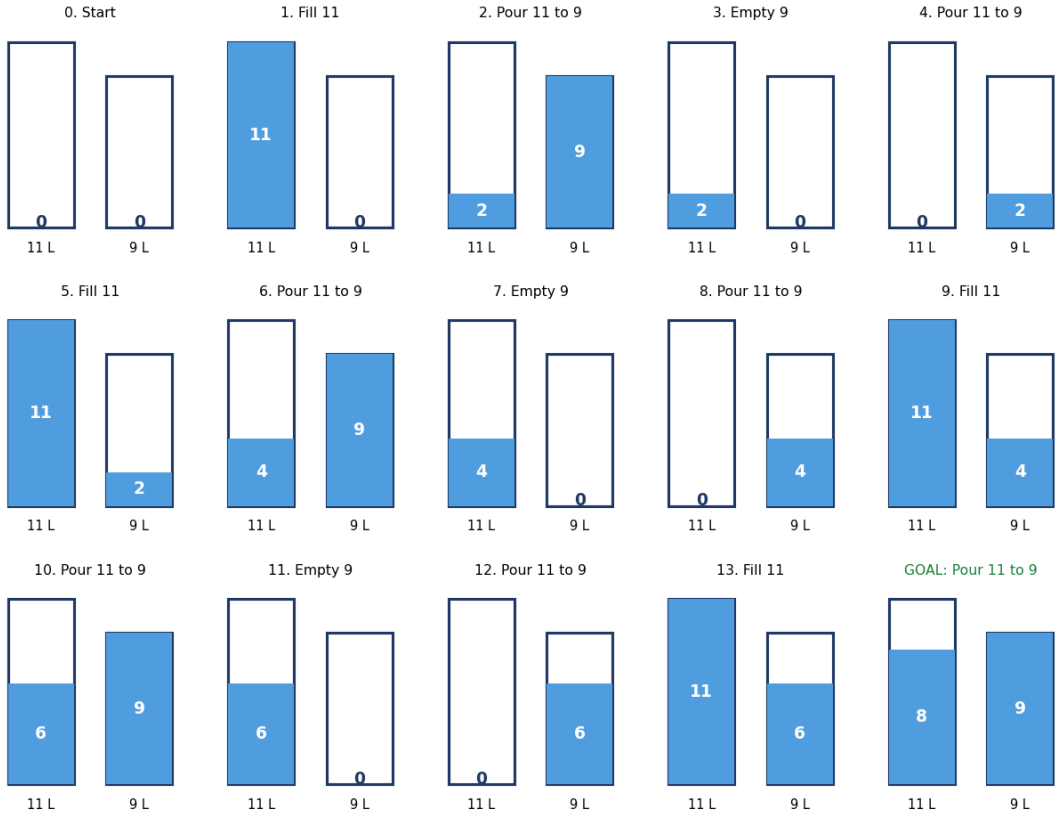


Figure 3.1: The 14 moves shown jug by jug (blue = water, numbers = litres).

If Level 19 on the website uses different capacities from the question, apply the same method: write states as (a, b), list the six actions, and run BFS.

Question 4: Connect Four AI Challenge

S.No	Report component	Answer
1	Title of the Game-Based Activity	Connect Four AI Challenge: defeat the AI opponent.
2	Game Platform / Link	Connect Four, https://www.mathsisfun.com/games/connect4.html
3	Objective of the Activity	Connect four of your discs in a row (horizontally, vertically or diagonally) before the AI does.
4	Problem Statement	Play against an AI opponent and plan moves carefully to win.
5	AI Concept / Domain	Adversarial search: Minimax with alpha-beta pruning and a heuristic evaluation function.
6	Initial State and Goal State	Initial state: empty 7-column, 6-row board. Goal state: four of your discs in a line (a win); a full board is a draw.

S.No	Report component	Answer
7	Actions / Operators Used	Drop a disc into any column that is not full; players alternate. A state is the board plus whose turn it is.
8	Solution Strategy	Think like the AI: use minimax, a two-player search where you pick the move that is best assuming the opponent also plays its best. Look several moves ahead and score positions by counting open lines of two and three discs for each player. Practical rules that follow from this search: • Start in the centre column (column 4); centre discs belong to the most lines. With perfect play the first player wins by starting in the centre (Allis, 1988). • Build double threats: a move that creates two different winning squares at once, because the opponent can block only one. • Block any opponent three-in-a-row immediately (look at squares that are both empty and playable). • Control threats by row parity: the first player aims for threats on odd rows (1, 3, 5), the second player on even rows (2, 4, 6). • Avoid dropping a disc that lets the opponent win on the square directly above it.
9	Step-by-Step Solution	The exact moves depend on the AI's replies, so a fixed move list cannot be given in advance. Recommended game plan: (1) play column 4; (2) if the AI replies in column 4, play column 3 or 5 to form two-in-a-row on the bottom row; (3) extend to an open three with both ends playable, which forces the AI to block; (4) use the forced replies to set up a second line through the same square, creating a double threat; (5) check every move for an immediate opponent win before playing it.
10	Game Performance	Number of games, result (win/draw/loss), moves and time: *[Fill in after you play on the site: number of attempts, time taken, moves, score.]*
11	Successful Completion and Evidence	Evidence of successful completion (winning board): *[Paste your own screenshot of the completed game here, showing the final board and the success message.]*
12	Efficiency and Optimality	Efficiency is measured in number of moves to win. Minimax with alpha-beta pruning examines about b^(d/2) positions instead of b^d (b = 7 moves, d = search depth), so deeper lookahead is possible in the same time. With perfect play the first player can force a win by the 41st disc placed overall (the 21st move of the first player) with perfect play; a win in fewer moves comes from forcing double threats early.
13	Analysis and Interpretation	The minimax model shows why defence and attack must be considered together: each of your moves is evaluated by the best reply the opponent has. The evaluation function turns an unfinished board into a score so the search can stop at a limited depth.
14	Critical Thinking and Alternative Solution	Stronger alternatives: iterative deepening with a transposition table; Monte Carlo Tree Search (MCTS), which plays thousands of random games from each candidate move; or a perfect solver

S.No	Report component	Answer
		based on the Allis knowledge-based approach. Move ordering (centre columns first) greatly increases pruning.
15	**Learning Outcome / Reflection**	**The game showed how an adversarial two-player game is modelled as a game tree and why lookahead plus heuristics gives strong play.**
16	**Conclusion**	**Applying minimax-based reasoning (centre control, forcing moves, double threats) gives the best chance to defeat the AI, which itself uses a similar search.**
17	**References**	**Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson (search chapters).** **Allis, V., A Knowledge-based Approach of Connect-Four, Master's thesis, Vrije Universiteit Amsterdam, 1988.** **https://www.mathsisfun.com/games/connect4.html**

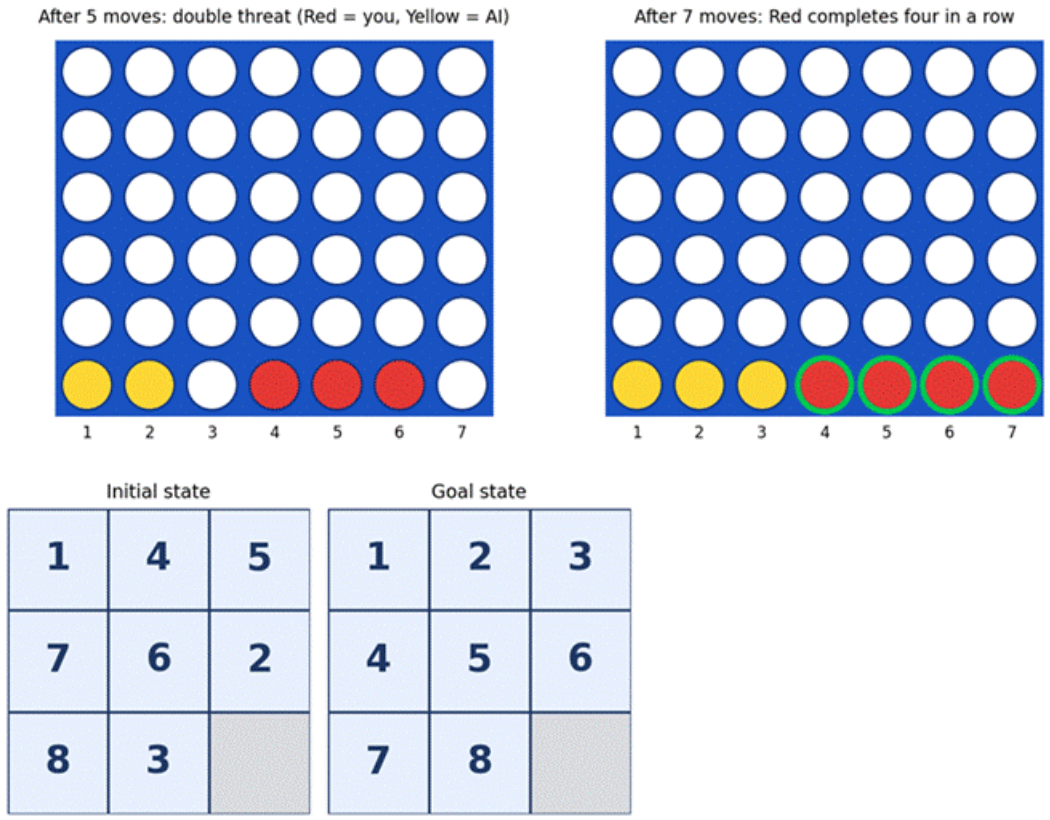


Figure 5.1: Sample initial and goal states.

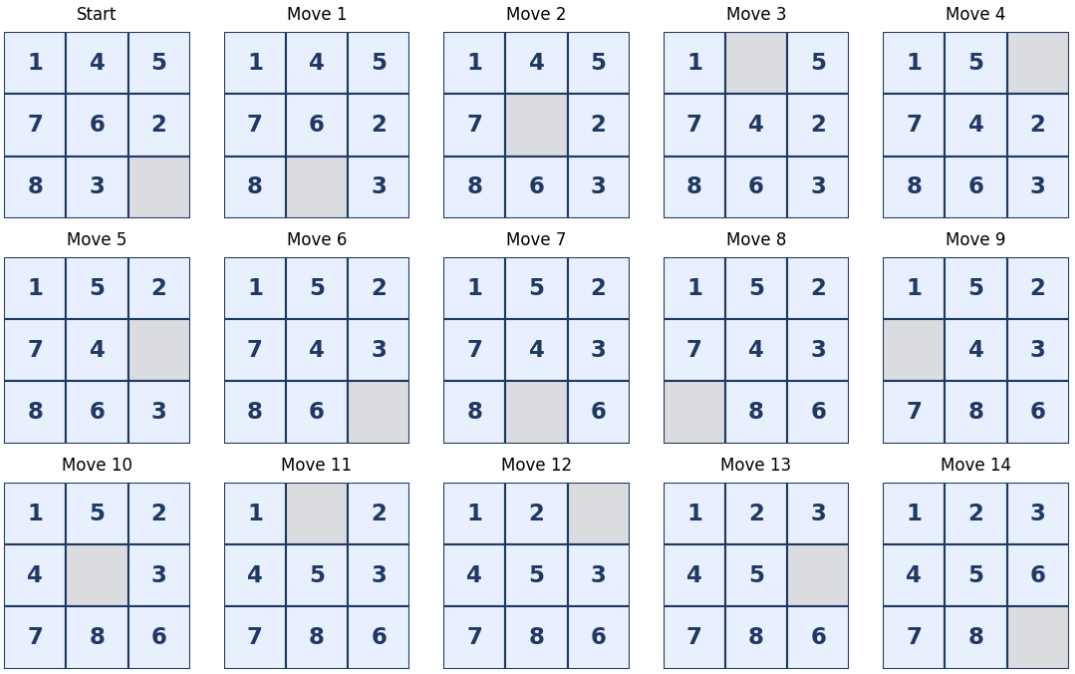


Figure 5.2: Optimal 14-move solution of the sample puzzle (state after each move).

Appendix A: 8-puzzle A* solver (enter your own board)

Change START to the board shown on the website (use 0 for the blank, read row by row). Run with Python 3.

```
import heapq
START = (1, 4, 5, 7, 6, 2, 8, 3, 0)      # <-- replace with your board
GOAL  = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def neighbours(s):
    z = s.index(0); r, c = divmod(z, 3)
    for name, dr, dc in (("Up",-1,0),("Down",1,0),("Left",0,-1),("Right",0,1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            l = list(s); l[z], l[nr*3+nc] = l[nr*3+nc], l[z]
            yield tuple(l), name, l[z]

def h(s):                           # Manhattan distance
    return sum(abs(i//3-(v-1)//3) + abs(i%3-(v-1)%3) for i, v in enumerate(s) if v)

def solvable(s):
    t = [v for v in s if v]
    return sum(a > b for i, a in enumerate(t) for b in t[i+1:]) % 2 == 0

def astar(start):
    pq = [(h(start), 0, start)]; g = {start: 0}; par = {start: None}
    while pq:
        f, gg, u = heapq.heappop(pq)
        if gg > g[u]: continue
        if u == GOAL:
            path = []
```

```
            while par[u]: u, name, tile = par[u]; path.append((tile, name))
            return path[::-1]
        for v, name, tile in neighbours(u):
            if v not in g or gg + 1 < g[v]:
                g[v] = gg + 1; par[v] = (u, name, tile)
                heapq.heappush(pq, (gg + 1 + h(v), gg + 1, v))

if not solvable(START): print("This board cannot be solved")
else:
    sol = astar(START)
    print(len(sol), "moves")
    for i, (tile, d) in enumerate(sol, 1): print(i, "slide tile", tile, "(blank moves", d + ")")
```

Appendix B: Maze BFS solver (for your own maze)

Write the maze as text: # for wall, . for open, S for start, G for goal. Replace the sample grid with your own.

```
from collections import deque
MAZE = ["S.#.....",
        ".##.###.",
        "....#...",
        ".####.#.",
        "......#G"]
R, C = len(MAZE), len(MAZE[0])
for r in range(R):
    for c in range(C):
        if MAZE[r][c] == "S": start = (r, c)
        if MAZE[r][c] == "G": goal = (r, c)

q = deque([start]); parent = {start: None}
while q:
    u = q.popleft()
    if u == goal: break
    for dr, dc in ((-1,0),(1,0),(0,-1),(0,1)):
        v = (u[0]+dr, u[1]+dc)
        if 0 <= v[0] < R and 0 <= v[1] < C and MAZE[v[0]][v[1]] != "#" and v not in parent:
            parent[v] = u; q.append(v)

if goal not in parent: print("No path")
else:
    path = []; u = goal
    while u: path.append(u); u = parent[u]
    path.reverse(); print("Shortest path =", len(path) - 1, "steps"); print(path)
```

Evaluation Rubrics

Criteria	Percentage (%)	Excellent	Good	Satisfactory	eds Improvem

Understanding of the Problem / Game Objective	10%	Clearly understands the problem and game objective	Understands the objective with minor errors	Partially understands the problem	Does not understand the objective
Application of AI Search / Problem-Solving Technique	15%	Correctly applies the appropriate AI search or problem-solving technique	Applies the technique with minor errors	Shows limited application of the technique	Unable to apply the appropriate technique
Successful Completion of the Game / Puzzle	20%	Successfully solves/completes the game independently	Completes the game with minimal assistance	Partially completes the challenge	Unable to complete the challenge
Efficiency of Solution / Optimal Moves	10%	Solves using optimal or highly efficient moves	Uses a reasonably efficient sequence of moves	Uses several unnecessary moves	Solution is highly inefficient or incomplete
Documentation / Evidence of Completion	15%	Provides clear steps, score, screenshots, and evidence of completion	Provides most required evidence	Provides incomplete steps or evidence	Provides no proper documentation or evidence
Analysis and Interpretation of the Solution	15%	Clearly analyses the solution and explains the AI technique and outcome	Provides a good analysis with minor gaps	Provides limited analysis or explanation	Unable to analyse or interpret the solution
Originality and Critical Thinking	15%	Demonstrates excellent independent thinking and explores an effective or alternative solution	Demonstrates good independent thinking	Shows limited critical thinking	Shows little or no independent thinking